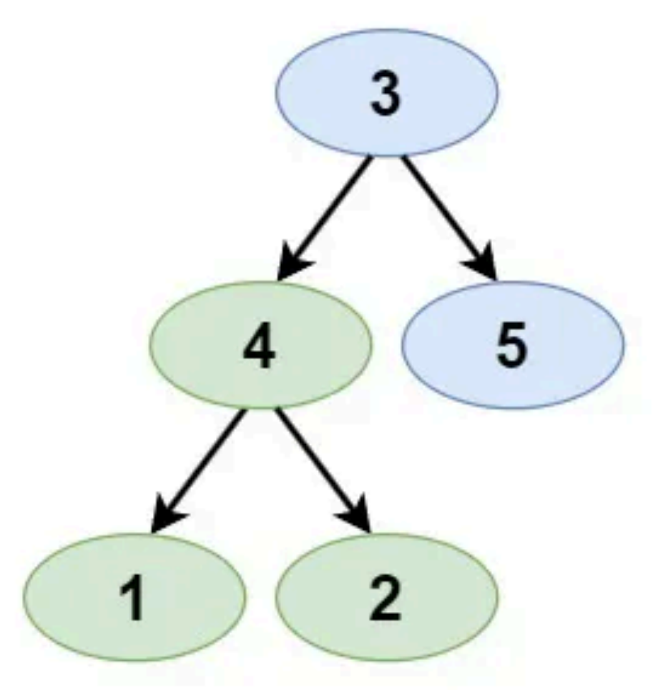
Solution: Subtree of Another Tree

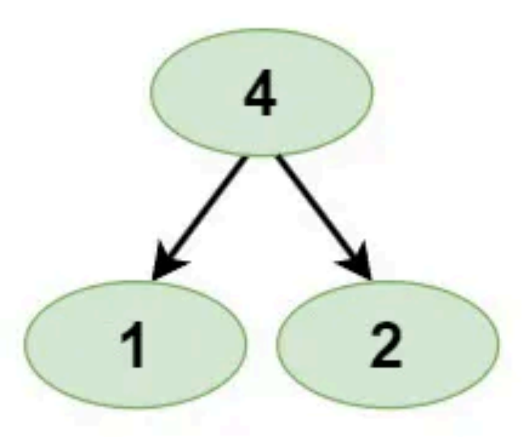
Problem Statement

Given two binary trees `s` and `t`, determine if tree `t` is a subtree of tree `s`. A tree `t` is considered a subtree of `s` if there exists a node in `s` such that the subtree of that node is identical to `t`. Both trees are considered identical if their structure and nodes are the same.

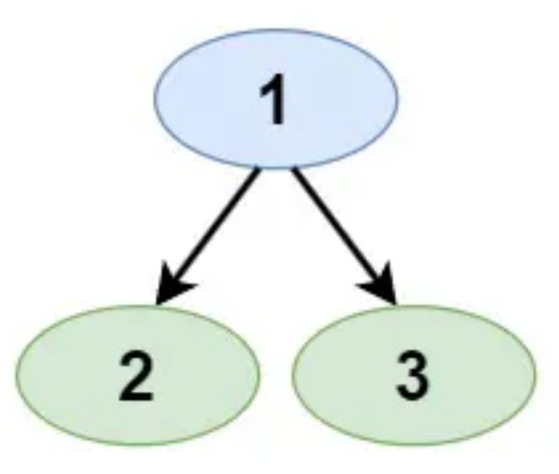
Examples

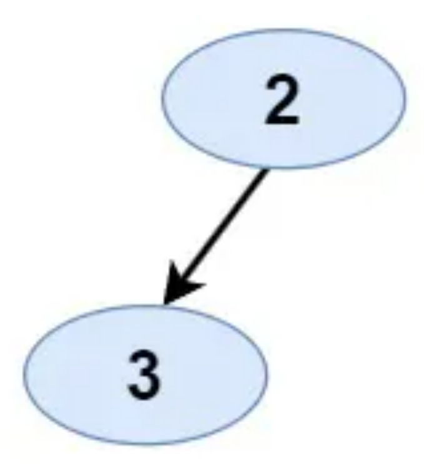
1.
    - **Input**:
        - Tree `s`: [3,4,5,1,2]
        - Tree `t`: [4,1,2]
    - **Expected Output**: true
    - **Justification**: Tree `t` can be found as a subtree of `s` rooted at the node with value 4.



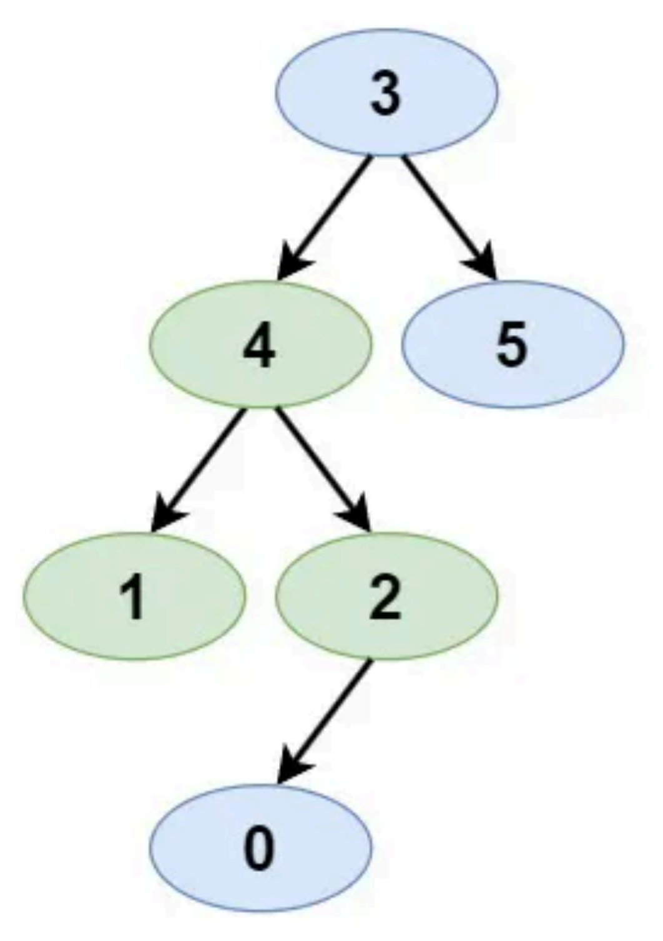


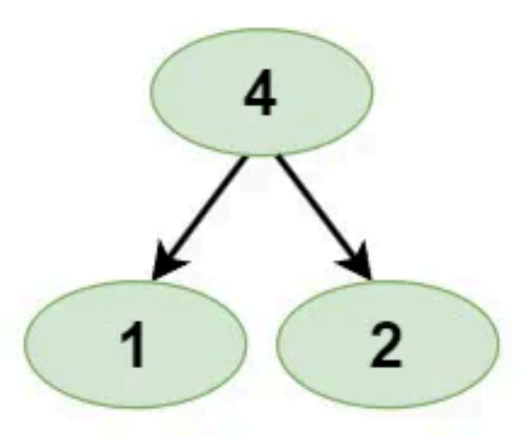
2.
    - **Input**:
        - Tree `s`: [1,2,3]
        - Tree `t`: [2,3]
    - **Expected Output**: false
    - **Justification**: There's no subtree in `s` that looks like tree `t`.





3.
    - **Input**:
        - Tree `s`: [3,4,5,1,2,null,null,null,null,0]
        - Tree `t`: [4,1,2]
    - **Expected Output**: false
    - **Justification**: Even though there's a subtree with root 4, it's not identical to tree `t`.





Solution

The central idea is to traverse through each node in tree `s` and for every node, attempt to match its subtree with tree `t`. If at any node, the subtree rooted at that node matches tree `t`, we have found our subtree, and we return true. If no such subtree is found after examining all nodes of `s`, we return false.

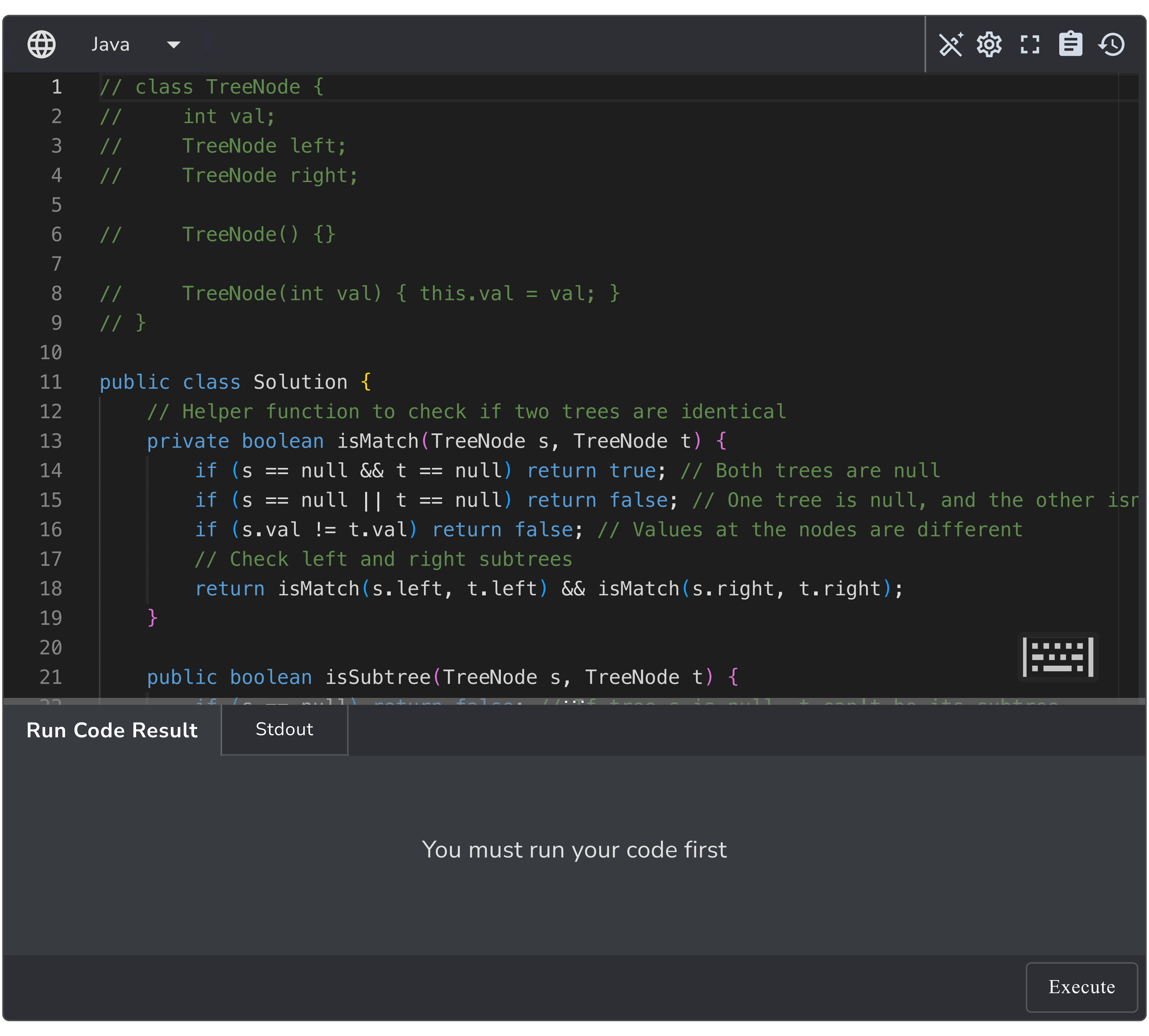
1. **Traversal and Comparison**:
    - As we traverse tree `s`, for every node, we try to see if the subtree rooted at that node matches tree `t`. To do this, we recursively compare nodes from the current subtree of `s` with tree `t`.
2. **Main Logic**:
    - We begin by checking if the current node of `s` matches the root of `t`. If so, we invoke the matching operation. If they turn out to be identical, then tree `t` is indeed a subtree of `s` rooted at the current node, and we return true.
    - If not, we continue the traversal by moving to the left and right children of the current node in `s`.
3. **Matching Operation**:
    - This operation checks if two trees are identical. We recursively compare nodes, starting from the given nodes in `s` and `t`. If the values of nodes being compared don't match, or if the tree structures differ, we return false. If all nodes match and the structures are identical, we return true.

Algorithm Walkthrough

Given Tree `s`: [3,4,5,1,2] and Tree `t`: [4,1,2]

- Start at the root of `s` (value 3)
    - `s` value doesn't match `t` root, so continue traversal.
- Move to the left child of `s` (value 4)
    - Now the value matches `t` root.
        - Compare left child of this node in `s` (value 1) with left child of `t` root (value 1).
            - They match.
        - Compare right child of this node in `s` (value 2) with right child of `t` root (value 2).
            - They match.
- Since both subtrees match, return true.

Code



```java
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;

//     TreeNode() {}

//     TreeNode(int val) { this.val = val; }
// }

public class Solution {
    // Helper function to check if two trees are identical
    private boolean isMatch(TreeNode s, TreeNode t) {
        if (s == null && t == null) return true; // Both trees are null
        if (s == null || t == null) return false; // One tree is null, and the other isr
        if (s.val != t.val) return false; // Values at the nodes are different
        // Check left and right subtrees
        return isMatch(s.left, t.left) && isMatch(s.right, t.right);
    }

    public boolean isSubtree(TreeNode s, TreeNode t) {
```

You must run your code first

Complexity Analysis

- **Time Complexity**:
    - In the worst case, we'll be checking each node of `s` to see if it's a subtree that matches `t`.
    - For each node of `s`, in the worst case, we'll be comparing it with each node of `t`.
    - Thus, the worst-case time complexity is O(m * n), where m is the number of nodes in `s` and n is the number of nodes in `t`.
- **Space Complexity**:
    - The space complexity is O(m) due to the recursion stack, where m is the depth of tree `s`.

← Previous
Subtree Of Another Tree (Easy)

Next →
Daily Temperatures (Medium)

☐ Mark as Completed